

Reporte de Agentic Loop

0. SKILLS CONFIGURADOS

arquitecto !' DISEÑO-ARQUITECTURA (GLX-DISEÑO-ARQUITECTURA-V1)
tech_lead !' MICROPLANIFICACION-DISEÑO (GLX-MICROPLANIFICACION-DISEÑO-V1)
desarrollador !' FRONTEND-DESARROLLO (GLX-FRONTEND-DESARROLLO-V1)
qa !' QA-VALIDACION (GLX-QA-VALIDACION-V1)
desarrollador_backend !' DESARROLLO_BACKEND-API (GLX-DESARROLLO_BACKEND-API-V1)

1. ANÁLISIS DE ARQUITECTURA

Micro-ADR: PROYEC-20 — Validador de Prompts y Tokens

1. CONTEXTO

El equipo de Ingeniería SDET requiere una utilidad interna de baja fricción para estimar tokens y latencia de prompts antes de integrarlos al orquestador principal. El requerimiento impone explícitamente una restricción de entrega de máximo 2 archivos (Single-File Architecture). No hay usuarios concurrentes masivos, no hay persistencia de datos, no hay autenticación, y el backend es una simulación matemática pura. El ciclo de vida esperado es una herramienta de desarrollo interno, no un producto de producción.

2. DECISIÓN ARQUITECTÓNICA

****Arquitectura elegida: Static File + Micro-Backend (2 archivos absolutos)****

- `index.html`: SPA autocontenida (HTML + CSS via Tailwind CDN + JavaScript vanilla con fetch API).
- `main.py`: Servidor FastAPI con un único endpoint POST `/api/analyze`, CORS habilitado para localhost, lógica de estimación matemática y simulación de latencia con `asyncio.sleep()`.

****NO se eligió:****

- 'L Monolito modular con estructura de carpetas (src/, routers/, models/, schemas/): YAGNI — un solo endpoint no justifica separación por capas.
- 'L Framework frontend (React, Vue, Svelte): YAGNI — la interactividad requerida es trivial (un formulario + fetch + render de 3 valores).
- 'L Base de datos (PostgreSQL, SQLite): No hay persistencia requerida en ningún criterio de aceptación.
- 'L Docker / docker-compose: Overhead innecesario para una herramienta de desarrollo local de 2 archivos.
- 'L Pydantic models en archivo separado: FastAPI permite inline models para casos de un solo endpoint.
- 'L Microservicios: No existe ningún equipo independiente desplegando partes separadas del sistema.
- 'L Sistema de colas (RabbitMQ, Redis): La 'latencia' es simulada con `asyncio.sleep()`, no hay procesamiento asíncrono real que justifique una cola.

3. JUSTIFICACIÓN BASADA EN YAGNI + KISS

El requerimiento técnico es explícito: 'máximo 2 archivos en total'. Cualquier archivo adicional (requirements.txt excluido como artefacto de dependencias, no de código) violaría la restricción del cliente. La lógica de negocio es: `tokens = word\_count \* 1.3` y `latency = random(500, 2000)`. Introducir capas de abstracción para esta lógica sería ingeniería especulativa pura. Tailwind via CDN elimina la necesidad de un pipeline de build (Webpack, Vite), reduciendo la fricción de setup a cero.

4. RIESGOS ASUMIDOS POR SIMPLIFICAR

- ****Riesgo 1 (Bajo):**** Tailwind CDN no es apto para producción (carga el CSS completo). Aceptado: es una herramienta interna de desarrollo.
- ****Riesgo 2 (Bajo):**** Sin `requirements.txt` versionado, la reproducibilidad depende del entorno del desarrollador. Mitigación: se incluye `requirements.txt` como artefacto de soporte (no cuenta como archivo de aplicación según la restricción).
- ****Riesgo 3 (Muy Bajo):**** La estimación de tokens por `palabras \* 1.3` es una aproximación gruesa vs. tokenizadores reales (tiktoken). Aceptado explícitamente en el requerimiento: 'cálculo matemático simple'.
- ****Riesgo 4 (Bajo):**** CORS abierto a localhost es inseguro en producción. Aceptado: scope es exclusivamente desarrollo local.

5. CONTRATO DE INTERFACES

- **Host Backend:** `http://localhost:8000`
- **Endpoint:** `POST /api/analyze`
- **Request Body:** `{ "prompt": "string" }`
- **Response Body:** `{ "estimated\_tokens": integer, "simulated\_latency\_ms": integer, "status": "Óptimo" | "Alerta" }`
- **Regla de negocio tokens:** `floor(word\_count \* 1.3)`, mínimo 1 si el texto no está vacío.
- **Regla de estado:** `status = 'Alerta'` si `estimated\_tokens > 1000`, sino `Óptimo`.
- **Latencia simulada:** `random.randint(500, 2000)` ms via `asyncio.sleep(latency / 1000)`.
- **CORS Origins permitidos:** `[http://localhost, 'http://127.0.0.1', 'null']` (el origen 'null' cubre apertura directa de archivo HTML en navegador).
- **Puerto frontend:** Apertura directa en navegador (file://) — sin servidor de archivos estáticos requerido.

2. MÉTRICAS DE ITERACIÓN (QA)

- index.html | Estado: APROBADO | Iteraciones: 1
- main.py | Estado: APROBADO | Iteraciones: 1
- requirements.txt | Estado: APROBADO | Iteraciones: 1

3. MÉTRICAS DE TOKENS IA

Tokens de entrada (input): 30,972

Tokens de salida (output): 8,387

Total tokens: 39,359

Horas-hombre estimadas: ~2.9 horas

Modelo utilizado: claude-sonnet-4-6

Desglose por Agente:

DISEÑO-ARQUITECTURA: !"2,147 input | !1,457 output | 1 llamadas

MICROPLANIFICACION-DISEÑO: !"7,335 input | !3,719 output | 3 llamadas

FRONTEND-DESARROLLO: !"9,695 input | !2,043 output | 3 llamadas

QA-VALIDACION: !"11,795 input | !1,168 output | 3 llamadas